

Tidy-up: GUI, workflow and feature review

Rename project and app TidyUp

My recommendation: make Tidy-up safe and predictable for everyday use before expanding its rule engine. The biggest opportunity is to turn it from a rule editor with file-monitoring capabilities into a complete workflow: configure folders, build a rule, see exactly what it will do, test it, enable it, and recover safely if something goes wrong.

I reviewed the public [Tidy-up repository](#), its GUI specification, the implementation-status documents, and the documented architecture. The project describes a Windows WPF application built around MVVM, SQLite, file monitoring, nested conditions and ordered file actions. Its documentation also reveals an important gap between the intended feature set and demonstrated end-to-end functionality.



GitHub

+2

One qualification: this is a repository/documentation review, not a locally executed build or a full runtime verification. I distinguish documented gaps from recommendations and areas that still need code-level validation.

1. Highest-priority findings

P0 — File-operation safety

Critical

Define and verify recovery, conflict handling, and partial-failure behaviour before enabling unattended automation. Moving or deleting the wrong file is more serious than a broken GUI binding.

P1 — Complete the user journey

High

Make rule creation, folder selection, simulation, review, saving, enabling and monitoring one coherent flow. The completion report explicitly describes the folder editor as a placeholder and the rule-test window as still needing UI work.



GitHub

+1

P1 — Verify the actual implementation

High

The repository contains several completion and fix summaries. Treat these as historical evidence, not proof that every advertised feature is currently wired up and tested.

Reconcile each claim with the source code and automated tests.

P2 — Improve day-to-day usability

Give users a clear dashboard, readable rule summaries, accessible status indicators, and a history view that explains what happened without requiring them to understand internal implementation details.

The key architectural principle should be: no rule should perform a consequential action until the user can understand its scope, preview its effects and configure its safety behaviour.

2. GUI fixes I would prioritise

P0 — Safety and operational feedback

These are requirements for trustworthy automation, not cosmetic improvements.

1. Add a genuine dry-run and impact review

Before execution, show the affected files, matched rule, condition results, ordered actions, final paths and conflict outcomes. Display counts for move, copy, rename, delete, skip and error.

Keep the actual execution button disabled until the plan has passed validation. Preview must not modify files or processed-file state.

2. Build a reliable undo and recovery experience

The specification calls for undo/history, but the implementation must be checked for actual restoration support. Store original and resulting paths, hashes, operation state and conflict details. Use the Recycle Bin for deletions by default; do not promise undo for permanent deletion.

3. Improve destructive-action safeguards

Require explicit confirmation for overwrite, permanent deletion and large batches. Add safeguards against moving a monitored folder into itself, source/destination overlap, recursive reprocessing and unintended rule chains.

For unattended operation, define a safe default for every conflict rather than waiting for an interactive dialog.

4. Make monitoring state visible

Show whether monitoring is running, paused or degraded; how many files are queued; the last successful scan; the last error; and whether watcher events were lost.

FileSystemWatcher events can be missed when its buffer overflows, so recovery must include a rescan rather than assuming every event arrived.

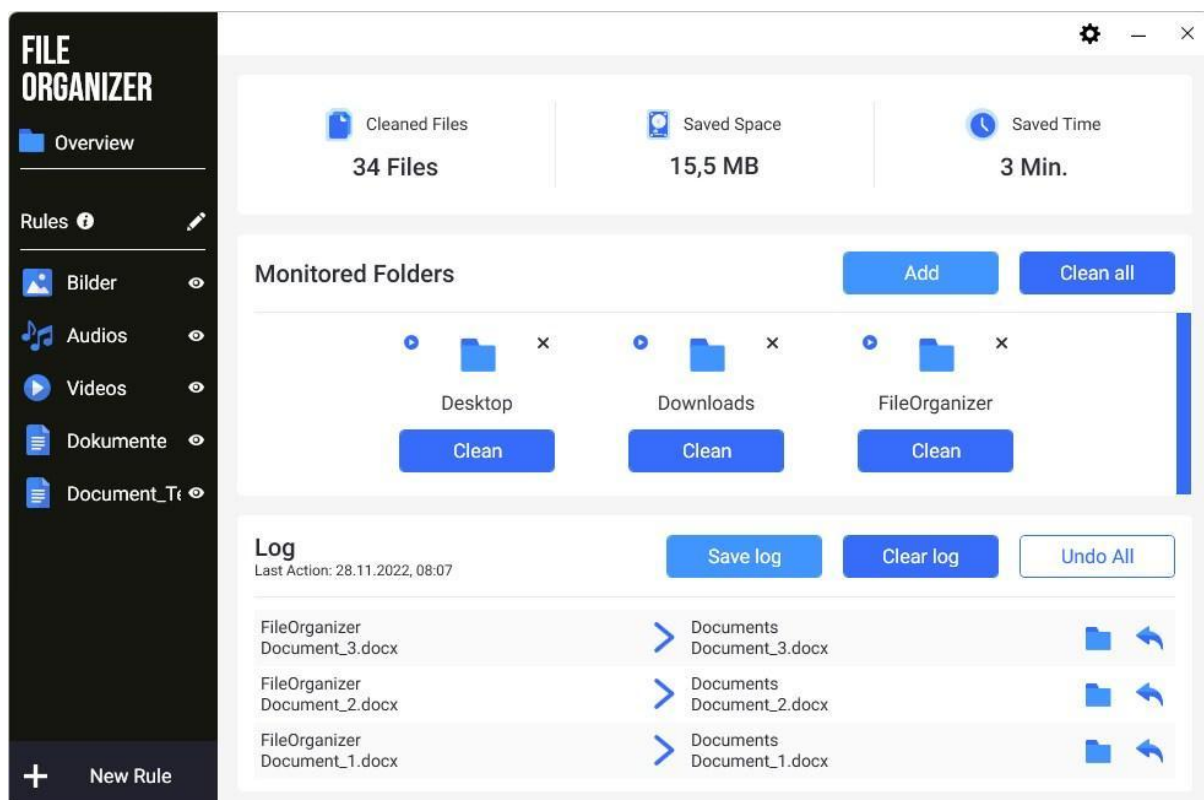
The existing specification already identifies dry-run previews, Recycle Bin defaults, confirmations, logging and undo as intended safety mechanisms. I would make their implementation and verification explicit acceptance criteria.



GitHub

+1

P1 — Make the interface easier to use



5. Replace the rule-editor-first experience with a dashboard

The initial screen should answer three questions: What is monitored? What needs attention? What happened recently?

Include active rules, watched-folder health, files processed today, pending warnings and the last execution result.

Create rule

Rule name
Artemis pictures

Trigger rule when
new article in folder Artemis

If article matches **all** any of the following

title or content contains Artemis

author is The_anonim_guy

URL doesn't match re... />https?:\V{/?;www\})...

+ Whole words only

Then do this

send to email Enter email address [Email settings](#)

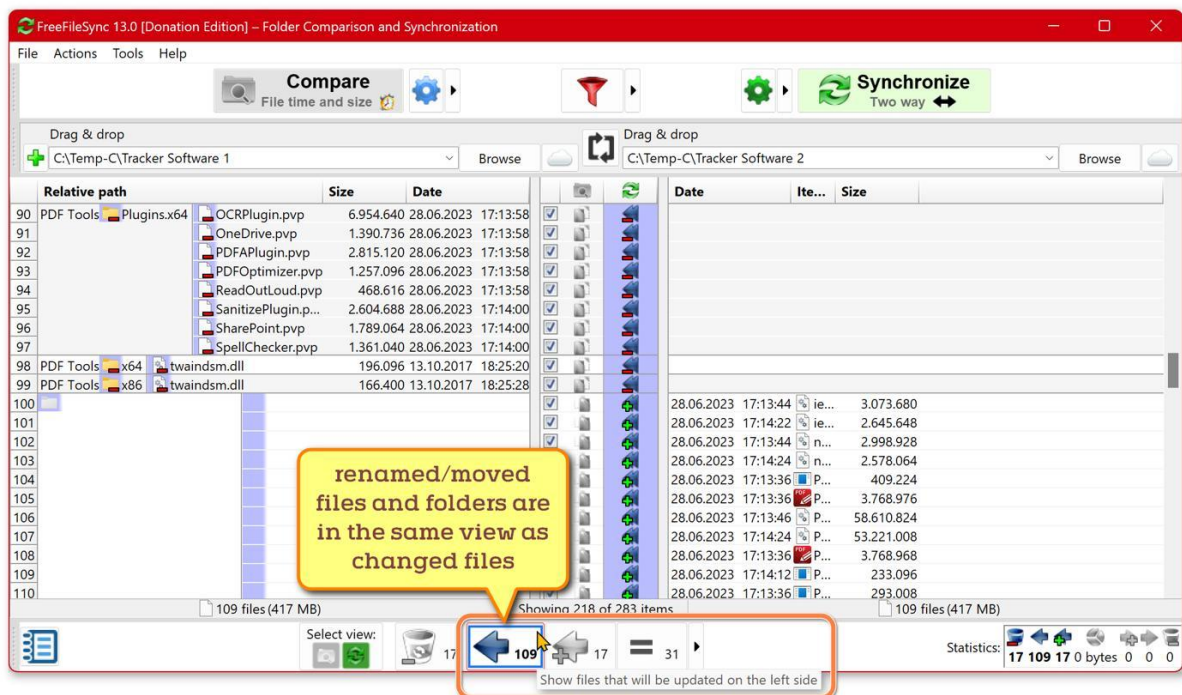
+

Run rule Save rule Cancel

6. Simplify rule creation

Use a guided editor with four stages: Folders → Conditions → Actions → Review. Keep advanced options available without exposing all of them at once.

Provide plain-language summaries such as “When a PDF is created in Downloads, move it to Documents/PDFs.”



7. Upgrade the preview and test screens

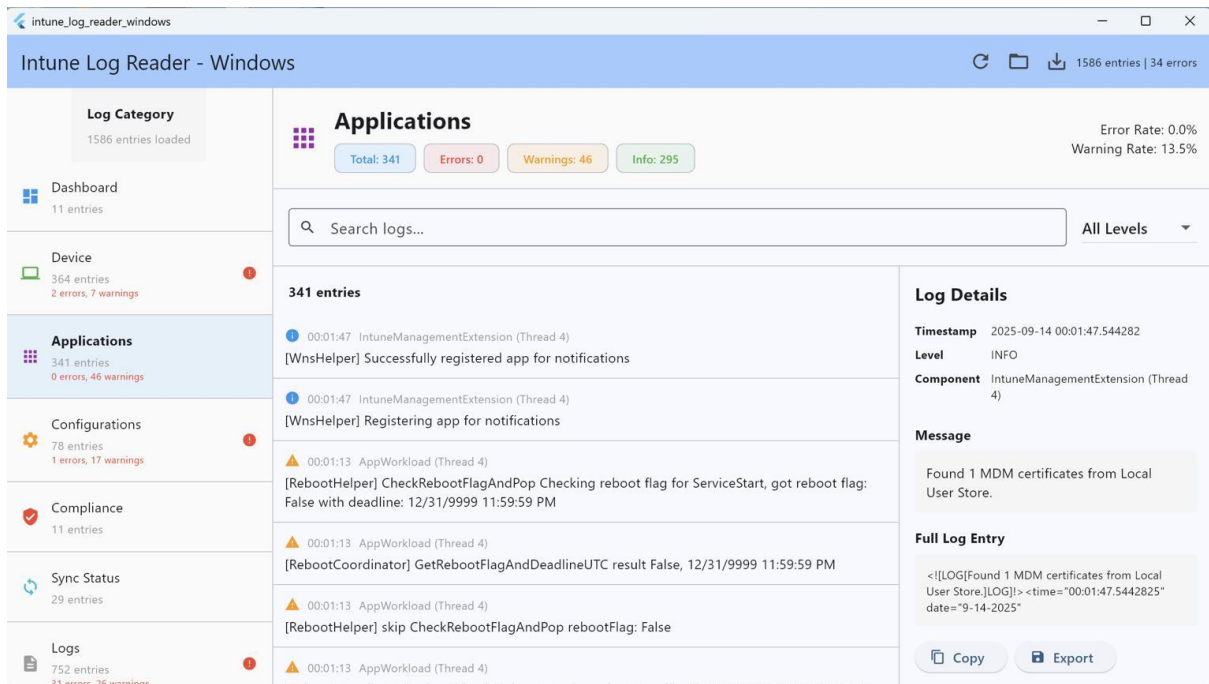
Display original and proposed paths side by side. Let users filter by rule, file type, result and error; inspect why a file matched; and test an individual action against a selected sample.

The project's own completion report says the test feature still needs its test-window UI, despite the toolbar feature being listed as available.



GitHub

+1



8. Improve the history screen

Make each operation expandable to show the original file, matched conditions, action sequence, timestamps, error details and recovery options. Support filtering and export without loading the entire log into memory.

These are proposed visual directions, not screenshots of the current application.

P2 — Polish and accessibility

- Use consistent spacing, typography, button placement and icons throughout the application.
- Add proper empty states: no rules, no monitored folders, no matching files, and no history.
- Mark unsaved changes and prompt before discarding them.
- Support keyboard navigation, accessible names, visible focus indicators and high-contrast themes.
- Avoid relying on red/green alone to communicate status.
- Replace the default maximised-only experience with a sensible, resizable window that remembers the user's preferred size.
- Add contextual help for regex, variable interpolation, conflict strategies and nested AND/OR logic.

The repository's GUI-fix report documents earlier binding, condition-template and help-system repairs. Those are useful foundations, but they do not establish that the entire user journey is complete.



GitHub

+1

3. The workflow I recommend

This should be the primary design target for the application.

Step 1 — Add monitored folders

Browse for folders, choose recursive scanning, configure exclusions and validate access.

Step 2 — Create or select a rule

Start from a template or create a custom rule. Show priority and enabled state.

Step 3 — Define conditions

Build nested AND/OR groups, validate inputs and inspect matching files.

Step 4 — Configure actions

Set destinations, variables, action order, conflict policies and failure behaviour.

Step 5 — Simulate the complete rule

Evaluate the rule against real files without changing the filesystem. Identify conflicts and dangerous paths.

Step 6 — Review and approve

Review the exact changes and counts. Require explicit approval before applying them.

Step 7 — Activate monitoring

Choose manual, scheduled or continuous execution. Show health, queue status and the last successful scan.

Step 8 — Review results and recover

Inspect successes and failures, retry safe operations, and restore supported changes from history.

Important workflow rules

1. A newly created rule starts disabled.
2. A new or materially edited rule requires a fresh preview before activation.
3. Saving a rule must not implicitly start monitoring or execute actions.
4. A preview is tied to the rule version and filesystem state it evaluated. Revalidate before applying if either has changed.
5. Rule ordering must be explicit, with clear handling when several rules match the same file.
6. Pausing monitoring must stop new work from being scheduled; already-running operations need a clearly defined completion or cancellation policy.
7. The UI must distinguish rule saved, monitoring active, operation running and operation completed. They are not the same state.

This workflow builds on the intended sequence in the [GUI specification](#), but makes the safety gates mandatory instead of optional UX details.



GitHub

+1

4. Missing features worth adding

I would prioritise features that improve reliability, explainability and recovery before adding more condition types or integrations.

Priority key: P0 = release blocker; P1 = core usability; P2 = valuable enhancement; P3 = optional expansion.

Priority Feature		Why it matters
P0	Transaction-aware file operations	Prevent partial action chains from leaving files in inconsistent locations.
P0	Conflict-safe rollback	Restore files without overwriting newer content or clobbering unrelated files.
P0	Rule validation and cycle detection	Catch recursive moves, invalid destinations and self-triggering rules.
P1	Rule templates	Let users start with common scenarios rather than building every rule manually.

P1	Match explanation	Show exactly which conditions passed or failed for each file.
P1	Reprocess and retry controls	Recover from locked files, temporary failures and missed events.
P1	Monitoring health and recovery scans	Detect watcher overflow, disconnected network shares and inaccessible folders.
P1	Rule import/export versioning	Make rule backups portable and provide safe schema migration between releases.
P1	Comprehensive integration tests	Test actual filesystem behaviour, not only individual methods or bindings.
P2	Scheduled rules and quiet hours	Avoid running expensive or disruptive jobs at inconvenient times.
P2	Notifications and system tray	Support unobtrusive background operation with visible pause/resume controls.
P2	Content-based conditions	Extend beyond filename, extension, size and dates to supported document metadata and text.
P2	Duplicate detection	Identify identical files using hashes before moving or copying them.
P2	Rule diagnostics	Explain why a rule did not run, why a file was skipped, or why a watcher stopped.
P3	Archive extraction and command actions	Extend the action engine after isolation, permissions and rollback behaviour are secure.
P3	Advanced variable and lookup management	Support reusable naming templates, lookup tables and validated custom variables.

The specification already lists some of these as intended capabilities, including custom variables, archive extraction, scheduled monitoring concepts and richer logging. The priority is to verify and finish the high-value items, not simply add more entries to the feature list.



GitHub

+2

Five features I would add beyond the current specification

1. Rule conflict analyser

Analyse enabled rules for overlapping conditions, competing destinations, duplicate actions, order dependencies and possible loops. Display warnings before activation.

2. Versioned rule history

Preserve previous rule versions, show a configuration diff and allow restoration. Each execution should record the exact rule version used, so later edits cannot rewrite the explanation of an earlier operation.

3. Preview versus actual-results comparison

Compare planned actions with executed actions. Explain changes in file state, destination conflicts, newly locked files and actions that were skipped or only partially completed.

4. Explainable file decisions

For every matched or rejected file, show the evaluated conditions, values, rule priority and chosen action. This makes the application much easier to debug than relying on a log entry saying only “matched.”

5. Disaster recovery and migration

Provide verified database backups, a restore procedure, database schema migrations, and a health check that detects inconsistent processing records. Keep database backup and file-operation recovery as separate mechanisms.

5. Technical areas to audit before expanding the GUI

These are the source-code checks I would give the implementation agent. They are especially important because the project uses filesystem watchers, EF Core/SQLite and ordered action execution.

A. File operation engine

- Confirm each action checks source existence, destination validity, permissions and conflicts immediately before execution.
- Test same-volume and cross-volume moves, locked files, long paths, duplicate names and network-share failures.
- Ensure a failed second or third action does not silently leave a partially completed workflow.
- Prevent a failed rollback from overwriting user modifications.

- Verify overwrite and delete semantics against the actual code, not the specification.

B. File watcher and scheduling

- Debounce event bursts and handle rename events correctly.
- Recover from watcher buffer overflow by scheduling a reconciliation scan.
- Prevent overlapping runs from processing the same file simultaneously.
- Exclude the application's own destinations when appropriate to avoid self-triggering loops.
- Use polling or explicit degraded-state reporting for network shares where notifications are unreliable.

C. Rule evaluation and data binding

- Verify nested AND/OR groups, empty groups, null values and invalid regex patterns.
- Ensure action order and rule priority persist correctly after save and reload.
- Test that edits to nested conditions, folders and actions reliably update the UI and persisted configuration.
- Check that importing older rules validates schema versions and handles unsupported fields.
- Confirm unsaved edits cannot be mistaken for the persisted rule used by the monitoring engine.

D. Database and logging

- Ensure migrations run safely and database backup restoration is tested.
- Use durable operation records that distinguish planned, running, completed, failed and recovery-required actions.
- Prevent logging failures from causing the UI to report false success.
- Use paginated history queries and retention policies that do not delete recovery information prematurely.

Testing standard

The repository's status documents report nine passing tests, but that historical number alone is not enough to establish release readiness.



GitHub

+1

I would add integration tests that use temporary directories and real files to exercise:

- Every action type in success, failure and conflict cases.
- Complete multi-action chains, including failure halfway through.
- Nested condition evaluation and rule ordering.
- Preview-to-execution consistency.
- Rollback after moves, renames and partial failures.
- File changes while a preview is open.
- Watcher overflow, duplicate events and network disconnection.
- Restarting the application during an operation.
- Rule import/export and database migration.
- GUI creation, editing, saving and reopening of a representative complex rule.

I would also make a clean Release build and the full test suite mandatory in CI. Where feasible, add UI automation for the main rule-creation journey.

6. Recommended implementation roadmap

0Phase 0 — Establish the baselineDetermine what actually works today.Priority: FirstBuild the current main branch in Release mode and run all tests.Trace each advertised feature from UI command to service and filesystem effect.Identify incomplete UI handlers, placeholder views and misleading status documents.Create a reproducible defect list with severity and regression tests.

1Phase 1 — Safety foundationMake file operations trustworthy.Priority: NextComplete preview and impact reporting.Harden conflict handling and path validation.Define partial-failure and rollback semantics.Verify watcher recovery and prevent overlapping work.Add filesystem integration tests for destructive operations.

2Phase 2 — End-to-end GUIMake the primary workflow complete.

3Phase 3 — Monitoring and diagnosticsMake background execution understandable.

4Phase 4 — Feature expansionAdd capability after reliability is established.

Do not begin with a large visual redesign. First establish which controls and services are actually functional, then complete the safety-critical paths. Otherwise, a more polished interface can conceal the same underlying reliability problems.

7. Repository and documentation improvements

There is a straightforward maintenance issue worth addressing alongside the code work.

The repository's [README.md](#) contains only the project heading, while the detailed GUI specification and multiple completion reports carry most of the explanation.



GitHub

+2

I would:

1. Replace the README with the actual purpose, supported Windows/.NET versions, installation and run instructions, screenshots, feature status, safety limitations and test commands.
2. Keep one authoritative roadmap instead of multiple documents claiming that the entire project is complete.
3. Mark each feature as Implemented, Partially implemented, Planned or Verified, with a link to its test evidence.
4. Add CI for Release builds, automated tests and dependency/security checks.
5. Add a small sample-rule library so users can evaluate the application quickly.
6. Document how to back up the SQLite database, recover interrupted operations and safely upgrade the application.

Final recommendation

My priority order would be:

1. Verify the code and tests against the advertised behaviour.
2. Make preview, conflict handling and recovery reliable.
3. Complete the folder → rule → simulation → approval → monitoring workflow.
4. Improve dashboard, diagnostics and history.
5. Add templates, scheduling and more advanced features.

The most valuable next deliverable is a source-verified Phase 0 audit that maps each GUI control and advertised feature to its implementation, test coverage, known defects and acceptance criteria. That gives you a reliable foundation for delegating the fixes to

Copilot or another coding agent without having it redesign working components or claim incomplete features are finished.